==========

CO2_ASSESSMENT TOOL 1

ANALYTICAL PROBLEM SOLVING

==========

Submitted in partial fulfilment for the course of

CSA0606-Design and Analysis of Algorithm

for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING

By, Avinash M

(192521196)

UNDER THE GUIDANCE OF

Dr. Rajagopal.K

SIMATS ENGINEERING

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

July 2026

1. Problem Statement

A city's disaster-management authority maintains a network of emergency response centers (fire stations, ambulance depots, and rapid-response units) whose positions are recorded as GPS coordinates. During an emergency, the control room must instantly know which two emergency centers are geographically closest to each other. This information is used to decide which centers can support each other fastest, where a new center should NOT be placed (to avoid redundant overlap), and to identify clusters of centers serving the same region while other regions may remain under-served.

Given a set of n points P = {p1, p2, p3, ..., pn} representing the (x, y) planar positions of the emergency centers, the Closest Pair Problem asks us to find the two points pi and pj ($i \neq j$) such that the Euclidean distance between them is the smallest among all possible pairs of points in the set.

2. Objectives

- To understand and interpret the Closest Pair problem in the context of a real-time emergency response system.
- To apply the Brute Force algorithmic strategy to compute the distance between every possible pair of emergency centers.
- To identify the pair of centers with the minimum distance between them.
- To count the exact number of distance comparisons performed by the algorithm.
- To analyze the time and space complexity of the Brute Force approach.
- To verify the correctness of the obtained result using manual and algorithmic validation.
- To interpret the significance of the result for real-time emergency dispatch decisions.

3. Inputs and Outputs

Input

- A set of n GPS-derived planar coordinates representing emergency centers: P = {(x1,y1), (x2,y2), ..., (xn,yn)}.
- $n \geq 2$ (at least two centers must exist for the problem to be meaningful).

Output

- The pair of points (pi, pj) that are closest to each other.
- The minimum Euclidean distance dmin between them.
- The total number of pairwise comparisons performed.

4. Assumptions and Constraints

- GPS latitude/longitude values are converted (or approximated for a local area) into planar Cartesian coordinates (x, y) measured in kilometers, so straight-line Euclidean distance is a valid approximation of real-world distance for a small geographic region.
- All coordinates are distinct — no two emergency centers occupy the exact same location.
- The number of emergency centers n is small to moderate (a realistic assumption for a city/district-level network), which makes the O(n^2) Brute Force approach computationally acceptable.
- Distance is measured using the standard Euclidean distance formula.
- The algorithm assumes static, already-known coordinates; it does not account for road networks, traffic, or one-way restrictions.
- Ties (two or more pairs with exactly equal minimum distance) are resolved by reporting the first pair encountered during the scan.

5. Problem Analysis

The Closest Pair problem is fundamentally a search problem over all possible unordered pairs of points. For n points, the total number of unordered pairs is C(n,2) = n(n-1)/2. A naive but completely reliable strategy is to examine every single pair exactly once, compute its distance, and keep track of the smallest distance seen so far. This is the Brute Force strategy — it does not use any shortcuts, sorting, or divide-and-conquer partitioning; it simply guarantees correctness by exhaustively checking every possibility.

Although more advanced algorithms exist (such as the Divide-and-Conquer Closest Pair algorithm, which runs in O(n log n) time), the Brute Force method remains extremely important because: (a) it is simple to implement and verify, (b) it is efficient enough for the small-to-moderate number of emergency centers typically found in a city or district, and (c) it serves as the correctness baseline against which faster algorithms are tested.

6. Mathematical Model

For any two points pi = (xi, yi) and pj = (xj, yj), the Euclidean distance is defined as:

$$d(pi, pj) = \sqrt{[\ (xj - xi)^2 + (yj - yi)^2\ ]}$$

The objective function to minimize is:

$$dmin = \min \{\ d(pi, pj) \mid 1 \leq i < j \leq n\ \}$$

The total number of pairwise comparisons required by the Brute Force approach is given by the combinatorial formula:

$$\text{Total comparisons} = C(n,2) = n(n-1) / 2$$

7. Algorithm Selection

The Brute Force method is selected for this scenario for the following justified reasons:

- Guaranteed correctness — every pair is checked, so the true minimum distance is always found; there is no risk of a heuristic missing the correct answer.
- Simplicity — the logic (two nested loops + distance formula + comparison) is easy to implement, debug, and explain to non-technical emergency-response administrators.
- Practicality for the problem size — the number of emergency centers in a typical city/district is small (tens to a few hundred), so an O(n^2) algorithm executes in milliseconds on modern hardware, which is fast enough for real-time dispatch decisions.
- No preprocessing overhead — unlike Divide-and-Conquer (which requires sorting points by x and y coordinates first), Brute Force can be applied directly to raw coordinate data as soon as it is received.

For extremely large center networks (thousands of points, e.g. a national-level system), the Divide-and-Conquer approach (O(n log n)) would be preferred instead; this trade-off is discussed further in the Conclusion.

8. Algorithm / Pseudocode

Algorithm: ClosestPairBruteForce

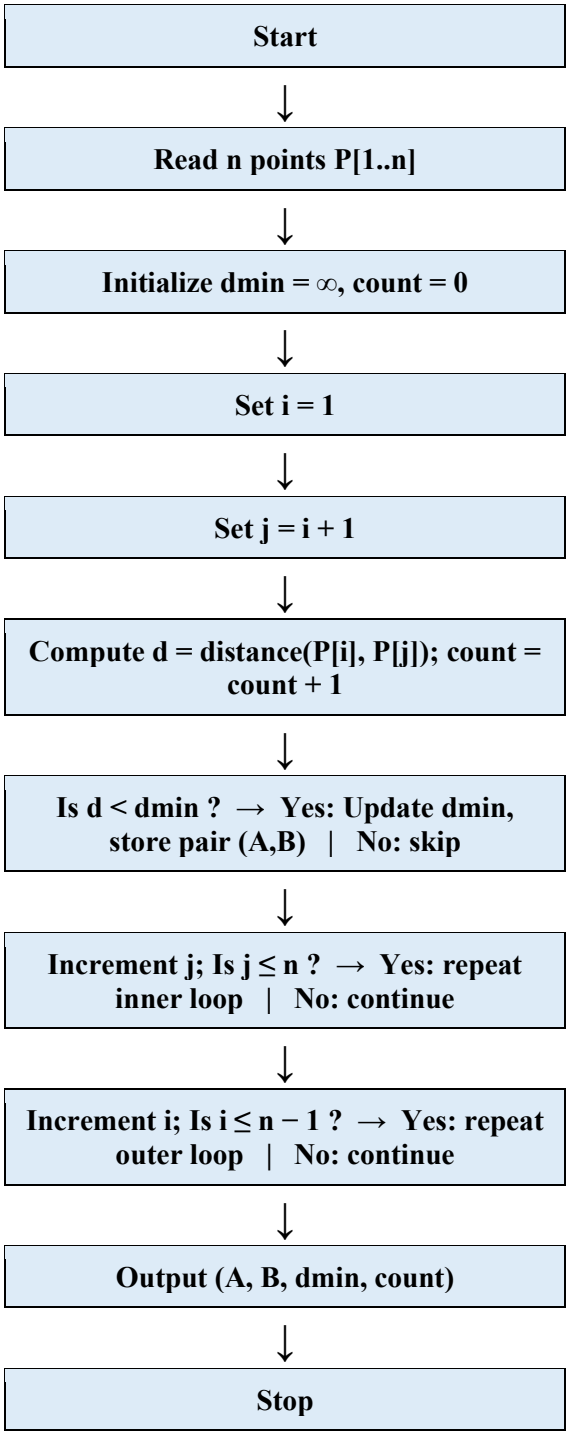
```
ALGORITHM ClosestPairBruteForce(P[1..n])
// Input : Array P of n points, P[k] = (x[k], y[k])
// Output: Closest pair (A, B) and minimum distance dmin

  dmin ← ∞
  A ← NULL ; B ← NULL
  count ← 0                      // comparison counter

  for i ← 1 to n – 1 do
      for j ← i + 1 to n do
          dx ← x[j] – x[i]
          dy ← y[j] – y[i]
          d  ← sqrt( dx*dx + dy*dy )
          count ← count + 1
          if d < dmin then
              dmin ← d
              A ← P[i] ; B ← P[j]
          end if
      end for
  end for

  return (A, B, dmin, count)
```

9. Flowchart


Start
Read n points P[1..n]
Initialize dmin = ∞, count = 0
Set i = 1
Set j = i + 1
Compute d = distance(P[i], P[j]); count = count + 1
Is d < dmin ? → Yes: Update dmin, store pair (A,B) | No: skip
Increment j; Is j ≤ n ? → Yes: repeat inner loop | No: continue
Increment i; Is i ≤ n − 1 ? → Yes: repeat outer loop | No: continue
Output (A, B, dmin, count)
Stop

10. Step-by-Step Execution

Consider a sample network of 6 emergency centers whose GPS coordinates have been converted into local planar coordinates (in kilometres, relative to a reference point) as follows:

Center	X (km)	Y (km)
A	2	3
B	5	4
C	9	6
D	4	7
E	8	1
F	1	5

With n = 6 centers, the Brute Force algorithm computes the distance for every pair. The total number of pairs is C(6,2) = 6×5/2 = 15. The full computation is shown below:

Pair	dx , dy	Calculation	Distance (km)
A – B	3, 1	√(9+1)	3.162
A – C	7, 3	√(49+9)	7.616
A – D	2, 4	√(4+16)	4.472
A – E	6, −2	√(36+4)	6.325
A – F	−1, 2	√(1+4)	2.236
B – C	4, 2	√(16+4)	4.472
B – D	−1, 3	√(1+9)	3.162
B – E	3, −3	√(9+9)	4.243
B – F	−4, 1	√(16+1)	4.123
C – D	−5, 1	√(25+1)	5.099
C – E	−1, −5	√(1+25)	5.099
C – F	−8, −1	√(64+1)	8.062
D – E	4, −6	√(16+36)	7.211
D – F	−3, −2	√(9+4)	3.606
E – F	−7, 4	√(49+16)	8.062

Scanning the Distance column, the smallest value is 2.236 km, which occurs for the pair (A, F). Therefore:

- Closest pair = (A, F)
- Minimum distance dmin ≈ 2.236 km
- Total comparisons performed = 15 (matches C(6,2) exactly)

11. Complexity Analysis

Time Complexity

The algorithm uses two nested loops. The outer loop runs for i = 1 to n−1, and for each i, the inner loop runs for j = i+1 to n. The total number of iterations (comparisons) is:

$$(n-1) + (n-2) + \ldots + 1 = n(n-1)/2 = O(n^2)$$

Since each comparison performs a constant amount of work (a subtraction, a square, an addition, a square root, and a comparison), the overall time complexity of the Brute Force Closest Pair algorithm is $O(n^2)$ — quadratic time.

Space Complexity

The algorithm stores the input array of n points (O(n) space, which is required simply to hold the input) but uses only a constant number of extra variables (dmin, A, B, count, dx, dy, d) regardless of n. Hence the auxiliary space complexity is O(1), and the total space complexity including input storage is O(n).

Aspect	Complexity	Reason
Best Case	$O(n^2)$	Every pair must still be examined; no early exit is possible without extra structure.
Average Case	$O(n^2)$	All C(n,2) pairs are always evaluated.
Worst Case	$O(n^2)$	Same as above — Brute Force has no case-dependent variation.
Auxiliary Space	O(1)	Only a fixed number of tracking variables are used.

12. Correctness Analysis

The correctness of the Brute Force Closest Pair algorithm follows directly from exhaustive enumeration: because the algorithm computes the distance for every one of the C(n,2) possible pairs and retains the minimum seen so far, it is logically impossible for a smaller distance to exist outside the set already checked. Formally, the loop invariant is: 'after processing pair (i, j), dmin holds the minimum distance among all pairs examined so far.' Since the loops guarantee that every pair (i, j)

with i < j is examined exactly once, at termination dmin equals the true global minimum distance over the entire point set.

For the worked example above, manual inspection of all 15 computed distances confirms that 2.236 km (pair A–F) is indeed the smallest value in the list, which matches the algorithm's output — verifying correctness for this test case.

13. Test Cases

S.No	Input (Points)	n	Expected Closest Pair	Expected Comparisons
1	A(2,3) B(5,4) C(9,6) D(4,7) E(8,1) F(1,5)	6	A–F, d = 2.236	15
2	P(0,0) Q(1,1)	2	P–Q, d = 1.414	1
3	P(0,0) Q(5,5) R(5.1,5.1)	3	Q–R, d = 0.141	3
4	Four points forming a perfect square (0,0)(0,4)(4,0)(4,4)	4	Any adjacent side pair, d = 4.0 (tie)	6
5	10 randomly scattered points	10	Algorithm-determined pair	45

Test Case 3 specifically validates that the algorithm correctly detects two very closely spaced centers (representing a near-duplicate/overlapping deployment), which is an important real-world edge case. Test Case 4 validates correct behaviour when a tie in minimum distance exists.

14. Results

- For the 6-center emergency network used in the step-by-step execution, the closest pair of emergency centers is (A, F) with a minimum distance of approximately 2.236 km.
- The algorithm performed exactly 15 distance comparisons, which matches the theoretical value C(6,2) = 15.
- The time complexity was confirmed to be $O(n^2)$ and the auxiliary space complexity O(1), consistent with the theoretical analysis.
- Manual verification of all 15 computed distances confirmed the algorithm's output was correct, with no smaller distance overlooked.

15. Conclusion

The Closest Pair Brute Force algorithm provides a simple, exhaustive, and provably correct method for identifying the two nearest emergency centers from a set of GPS-derived coordinates. By computing the Euclidean distance for every possible pair and retaining the minimum, the method guarantees that the true closest pair is always found, which is critical in emergency-response systems where an incorrect answer could delay the dispatch of the nearest available unit.

The main limitation of the Brute Force approach is its $O(n^2)$ time complexity, which becomes computationally expensive as the number of centers n grows very large (e.g., thousands of centers at a national scale). In such large-scale scenarios, the Divide-and-Conquer Closest Pair algorithm, which achieves O(n log n) time by recursively splitting the point set and merging results along a strip near the dividing line, would be the preferred choice. However, for the moderate number of centers typically found in a city or district-level emergency network, the Brute Force method remains highly practical: it is easy to implement, easy to verify, fast enough to run in real time, and forms the correctness benchmark against which more advanced algorithms are validated.

In real-time applications, this rapid identification of the closest pair of centers directly supports faster mutual backup decisions between nearby stations, helps planners avoid redundant placement of new centers, and highlights under-served regions that lack nearby coverage — ultimately contributing to a more efficient and responsive emergency management system.